UNIT- IV
Unsupervised Learning

- Dr. K. K. Bhoyar
Professor of IT

Contents

- k-Means Clustering,
- Choosing the Number of Clusters
- Hierarchical Clustering: agglomerative and divisive clustering, dendrogram,
- Semi supervised learning

Unsupervised Learning

- Unsupervised learning algorithms involves training with unlabeled examples
- K-means algorithm
- No prediction as in regression or classification
- Two types:
  - Clustering: Grouping similar instances
  - Learning Associations: discover interesting relationship hidden in large datasets
- Example applications
  - Customer segmentation in CRM
  - Image Segmentation
  - document clustering based on topics, etc.
  - Market Basket Analysis

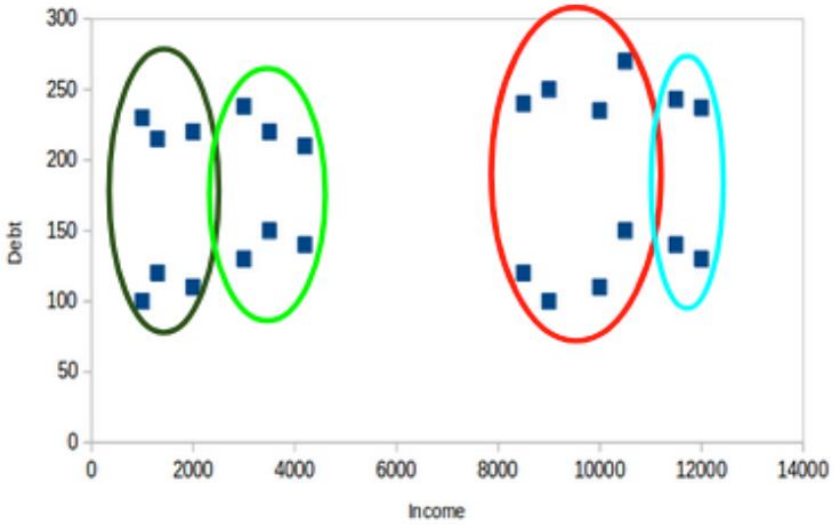
k-means Clustering

- It is an iterative algorithm that tries to partition the dataset into K pre-defined distinct *non-overlapping subgroups (clusters)* where each data point belongs to only one group
- K-means tries to partition x data points into the set of k clusters where each data point is assigned to its closest cluster.
- This method is defined by the objective function which tries to *minimize* the sum of all squared distances within a cluster, for all clusters.

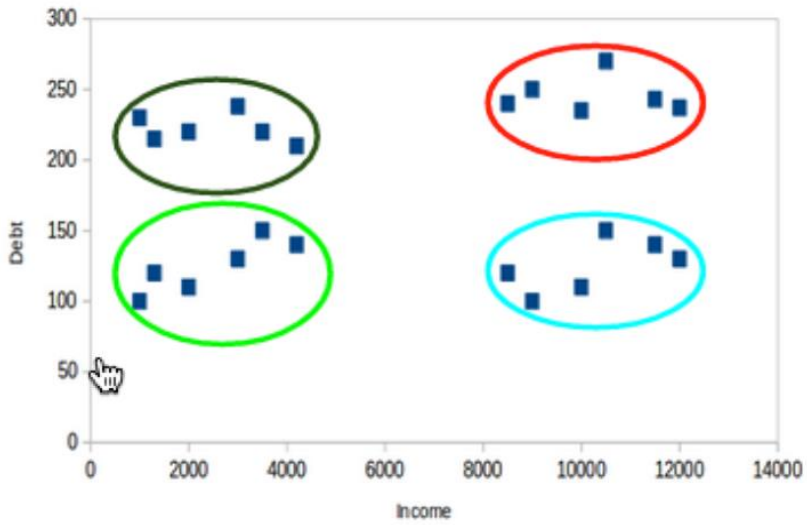
k-means Clustering…

- Objective of any clustering algorithm is to minimize the *intra-cluster* distances and maximize the *inter-cluster* distances
- cluster's *centroid* is the arithmetic mean of all the data points that belong to that cluster
- The algorithm assigns data points to a cluster such that the *sum of the squared distances* between the data points and the cluster's centroid is at the minimum.
- The less variation we have within a clusters, the more homogeneous (similar) the data points are within that cluster.

Which clustering is better for k=4 ?



Case - I



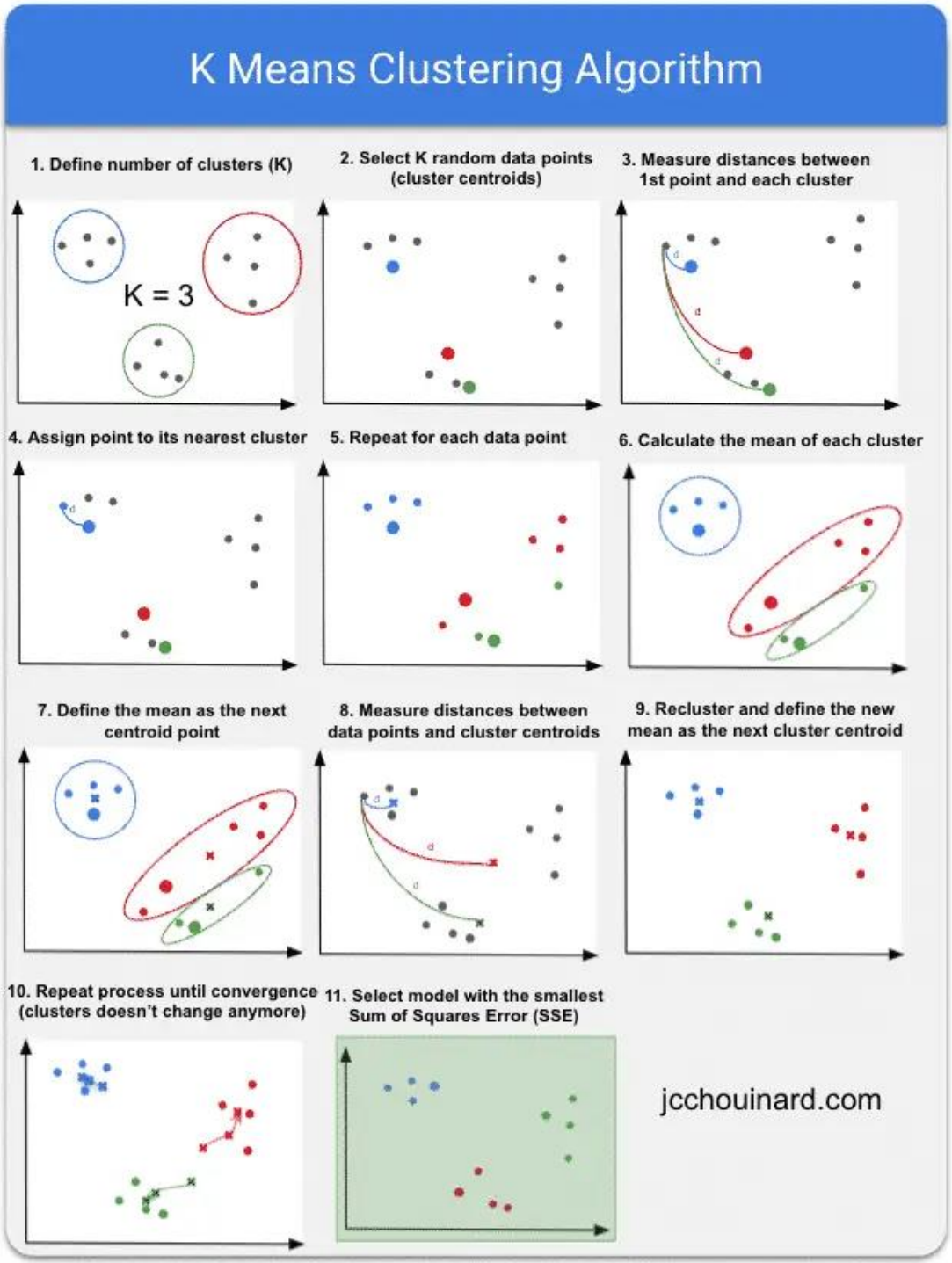
Case - II

We need "compact and well separated clusters".

Algorithm: k-means Clustering

Input to this algorithm are data points represented by feature vectors $\mathbf{x_1}, \mathbf{x_2}, \ldots \mathbf{x_n}$ and value of k. The steps of this algorithm are:

1. Pick k random points as cluster centres called centroids.
2. Assigns each data point $\mathbf{x_i}$ to the nearest centroid.
3. Recompute the centroids $\mathbf{m_i}$ as the mean of the data points assigned to each cluster.
4. Repeats steps 2 and 3 until the cluster assignments stabilize or a maximum number of iterations is reached.


K Means Clustering Algorithm
1. Define number of clusters (K)
K = 3
2. Select K random data points (cluster centroids)
3. Measure distances between 1st point and each cluster
4. Assign point to its nearest cluster
5. Repeat for each data point
6. Calculate the mean of each cluster
7. Define the mean as the next centroid point
8. Measure distances between data points and cluster centroids
9. Recluster and define the new mean as the next cluster centroid
10. Repeat process until convergence (clusters doesn't change anymore)
11. Select model with the smallest Sum of Squares Error (SSE)
jcchouinard.com

k-means Clustering Algorithm

- Here $\mathbf{x}^t$ are training examples and
- $\mathbf{m}_i$ are cluster centroids
- b_i^t *are estimated labels*;
- if b_i^t is 1, we say that $\boldsymbol{x}^t$ belongs to the group of $\boldsymbol{m}_i$.

Initialize $\boldsymbol{m}_i, i = 1, \ldots, k$, for example, to k random $\boldsymbol{x}^t$
Repeat
  For all $\boldsymbol{x}^t \in \mathcal{X}$

$$b_i^t \leftarrow \begin{cases} 1 & \text{if } \|\boldsymbol{x}^t - \boldsymbol{m}_i\| = \min_j \|\boldsymbol{x}^t - \boldsymbol{m}_j\| \\ 0 & \text{otherwise} \end{cases}$$

  For all $\boldsymbol{m}_i, i = 1, \ldots, k$

$$\boldsymbol{m}_i \leftarrow \sum_t b_i^t \boldsymbol{x}^t / \sum_t b_i^t$$

Until $\boldsymbol{m}_i$ converge

- Assign x^t to the cluster center with minimum distance
- Update k cluster centers by averaging the points in each cluster

Example:

How many bytes will be required in a byte addressable machine to keep track of cluster assignments for k = 8 and 100 training examples ? Assume that the one hot encoding is used to encode labels.

Before and After Clustering(K=4)

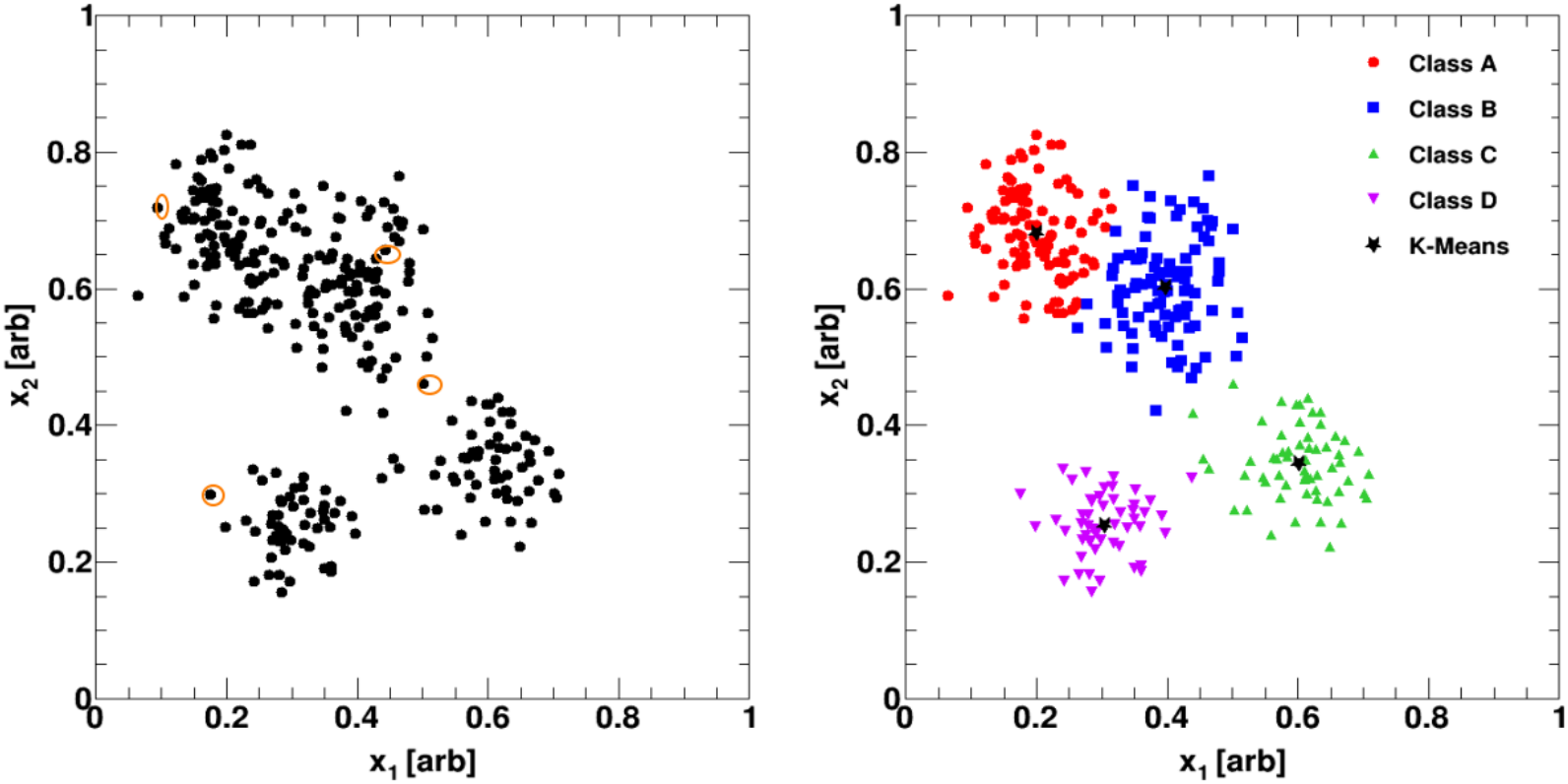
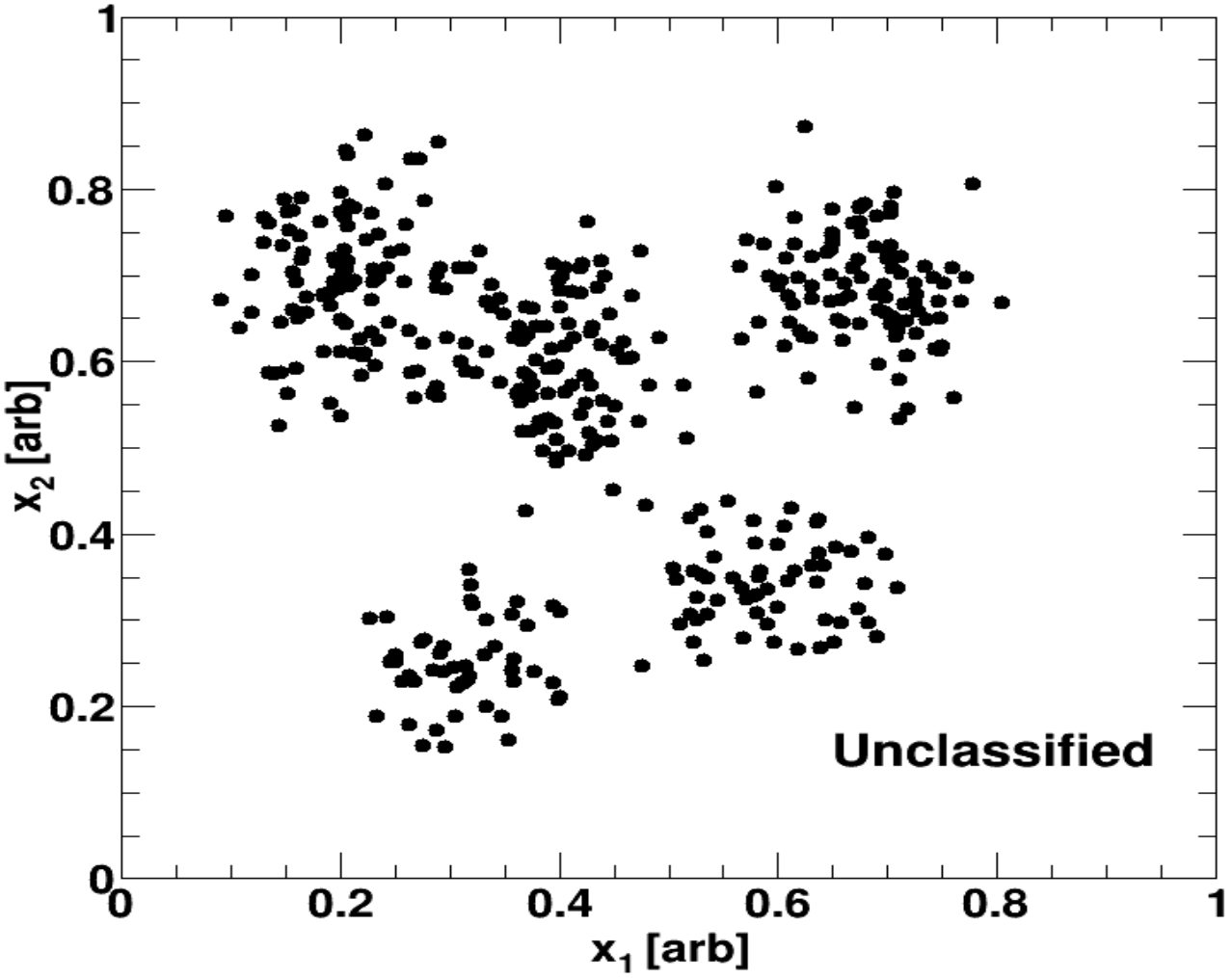


Fig. - Unlabeled data (left) with intial cluster centers, is clustered or grouped into 4 categories (right), which are arbitrarily labeled as A, B, C, and D.
(Source:https://freddyox.github.io/blog/Kmeans/)

K-means in Action(K=5)



(Source:https://freddyox.github.io/blog/Kmeans/)

Reflection:

1. What is the primary computational outcome of the K-Means clustering algorithm?

A. set of decision trees for classification

B. A trained K-means model

C. A set of cluster assignments and cluster centroids

D. A probability distribution over classes

2. In K-Means clustering, the final cluster centroids are computed as:

A. The mode of the data points in each cluster

B. The weighted sum of all data points

C. The mean of all data points assigned to the cluster

D. The median of all data points assigned to the cluster

Example on k-means clustering

Assume, you want to cluster 7 observations into 3 clusters using K-Means clustering algorithm : **{(1,1), (4,4), (7,7),(0,4), (4,0), (5,5), (9,9)}.**
Suppose after first iteration the clusters C1, C2, C3 has the following observations:
C1: {(1,1), (4,4), (7,7)}, C2: {(0,4), (4,0)}, C3: {(5,5), (9,9)}.
What will be the cluster centroids after the third iteration?

ANS:
Given:
1. After first iteration the given cluster assignments as follows:
C1: {(1,1), (4,4), (7,7)}, C2: {(0,4), (4,0)}, and C3: {(5,5), (9,9)}.
Now the cluster centroids for the second iteration are found by taking mean of observations in each of the three clusters.
Hence Centroids for cluster C1,C2,C3 becomes :
m1=(4,4), m2=(2,2) and m3=(7,7) respectively.

2. After the second iteration the clusters assignments (based on calculated distances) are: C1: {(4,4),(5,5) }, C2: { (1,1),(0,4),(4,0)}, C3: {(7,7),(9,9) }
Hence Centroids C1,C2,C3 becomes :
m1=(4.5,4.5), m2=(1.67,1.67) and m3=(8,8) respectively.

3. Find centroids after third iteration. Shall we continue the iterations ?

Case Study-I: Customer Segmentation for Supermarket

Problem Statement:

- BB supermarket mall maintains the customer data using membership cards. They have some basic data about the customers like Customer ID, name, age, gender, number of visits to the supermarket in a year, and the average spending per visit.
- They want to understand their customers so that the sense can be given to the marketing team so that they can plan the strategy accordingly. As a data scientist your task is to identify the customer groups and interpret them appropriately after analysing the clusters.
- This can be later given to the marketing team to increase the customer base and the profit of the supermarket.

Dataset

Dataset Description:

The BB Supermarket dataset consists of customers like Customer ID, name, age, gender, number of visits to the supermarket in a year(Nvisits), and the average spending per visit(purchaseAmount).

Out of these Nvisits and purchaseAmount can be used for categorization of customers, rest do not play a very important role.

Groups (k)	4
Samples Total	16
Dimensionality	2
Features	ID, name, age, gender, Nvisits, purchaseAmount

Handling Sensitivity to Initial Centroids

Clever Initialization: K-Means++

- **K-Means++** is an improved initialization method that helps in better cluster assignments. Instead of picking initial centroids randomly, it follows these steps:
  - **Pick the first centroid randomly** from the data points.
  - **For each remaining centroid**, choose a new point with a probability proportional to its squared distance from the nearest existing centroid.
- This ensures new centroids are spread out rather than clustered together giving faster convergence.
- **Repeat until K centroids are selected.**
- **Run the standard K-Means algorithm** with these initial centroids.

What is Scikit-learn ?

- Scikit-learn is an open-source Python library for machine learning.
- The library supports state-of-the-art algorithms such as KNN, XGBoost, random forest, SVM among others.
- It is built on top of Numpy. (NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.)
- Scikit-learn is widely used in prominent tech companies.
- **Provision for K-means ++ in sklearn**

```
# KMeans with k-means++ initialization (default) else use 'random'
kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)
kmeans.fit(X)
```

Python Implementation for BB Supermarket (K=4)

```
#BB Supermarket Customer Segmentation using K-means Clustering
#Using The sklearn(scikit-learn) package

import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# X contains training data samples
X=[[3,280],[5,320],[8,80],[2,225],[5,80],[6,265],[1,71],[5,235],[7,98],[4,300],[2,210],

# Number of clusters k
k=4
kmeans = KMeans(n_clusters=k)

# Fitting the input data
kmeans = kmeans.fit(X)

# Centroid values
Xarr=np.array(X)
centroids = kmeans.cluster_centers_
print("\nCluster Centers are :\n",centroids)

# Predicting the cluster labels
labels = kmeans.predict(X)
print("\nCluster Labels are:\n",labels)

colors = ['r', 'g', 'b', 'y', 'c', 'm']

fig, ax = plt.subplots()
for i in range(k):
        points = np.array([X[j] for j in range(len(X)) if labels[j] == i])
        ax.scatter(points[:, 0], points[:, 1], s=7, c=colors[i])
        ax.scatter(centroids[i, 0], centroids[i, 1], marker='*', s=200, c=colors[i])

# predicting class of input pattern [4,200]
xnew=[[4,200]]
print("\nPattern ", xnew, "belongs to cluster #",kmeans.predict(xnew))
```

Cluster Interpretation Strategies

- **Profile each cluster using centroid values**
- **Compare across clusters** to identify what makes each segment unique
- **Label clusters** meaningfully (e.g., “High Value”, “Occasional Buyers”, etc.)
- **Develop targeted strategies** (personalized marketing, promotions, etc.)

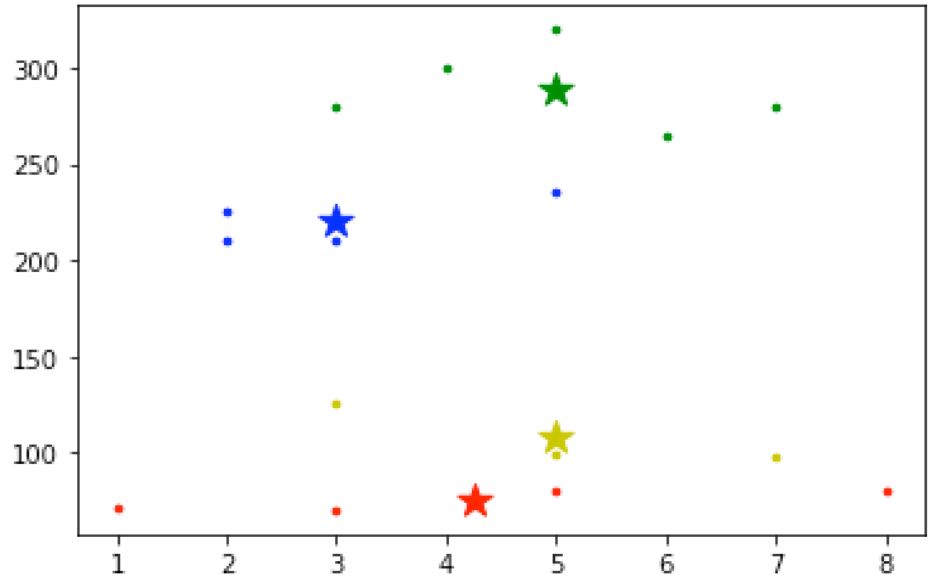
Interpreting Output

```
In [91]: runfile('/Users/apple/BB_kmeans', wdir='/Users/apple')

Cluster Centers are :
 [[  4.25         75.25      ]
 [  5.          289.        ]
 [  3.          220.        ]
 [  5.          107.33333333]]

Cluster Labels are:
 [1 1 0 2 0 1 0 2 3 1 2 0 3 2 3 1]

Pattern  [[4, 200]] belongs to cluster # [2]
```




```
In [92]:
```

Data Scientist may label these segments as:

Window Shoppers

Basic customers

Moderate buyers

Prime customers

Disadvantages of k-means clustering

1. Sensitivity to Initial Centroids (Initialization Can Impact Results)
2. Dependence on the Number of Clusters (k)
3. Assumes Spherical and Equal-Sized Clusters
4. Sensitive to Outliers
5. Requires Pre-processing(sensitive to the scale of features)

Customer Segmentation – Application of k-means

- Customer Segmentation is the subdivision of a market into discrete customer groups that share similar characteristics.
- Customer Segmentation can be a powerful means to identify unsatisfied customer needs.
- Using the above data companies can then outperform the competition by developing uniquely appealing products and services.
- The most common ways in which businesses segment their customer base are:
  - **Demographic information**, such as gender, age, familial and marital status, income, education, and occupation.
  - **Geographical information**, which differs depending on the scope of the company. For localized businesses, this info might pertain to specific towns or counties. For larger companies, it might mean a customer's city, state, or even country of residence.
  - **Psychographics**, such as social class, lifestyle, and personality traits.
  - **Behavioural data**, such as spending and consumption habits, product/service usage, and desired benefits.

Case Study-2:
Customer Segmentation for Mobile Customers

It is required to identify the groupings among domestic customers of ABC Telecomm Service provider company based on monthly phone call duration(Hours) and data consumed(GB) by each customer. These grouping will be useful for the company to have insight into their usage behaviour to offer better services and maximize company profit.

- Range of monthly calls [0 200] Hrs
- Range of monthly Data Consumed [0 200]GB
- Given the dataset , write a program to find the groupings in data (customer segments).
- As a data scientist how do you interpret the groups so obtained?
- Are you able to suggest standard data plans for the customer so that the company profit is increased ?

Performance measures for unsupervised learning

1. Within Cluster Sum of Squares (WCSS)
   - also known as Inertia. It measures the compactness of clusters, and it is calculated as the sum of squared distances between data points and their assigned cluster centroids.
   - Lower inertia indicates better clustering.
2. Silhouette Score:
   - The silhouette score measures how similar each data point is to its own cluster compared to other clusters.
   - It ranges from -1 (a poor clustering) to +1 (a perfect clustering). A higher silhouette score is desirable.
3. Dunn Index:
   - The Dunn index assesses the compactness and separation of clusters.
   - It is calculated as the ratio of the minimum inter-cluster distance to the maximum intra-cluster distance. A higher Dunn index is better.

Within-Cluster Sum of Squares (WCSS)

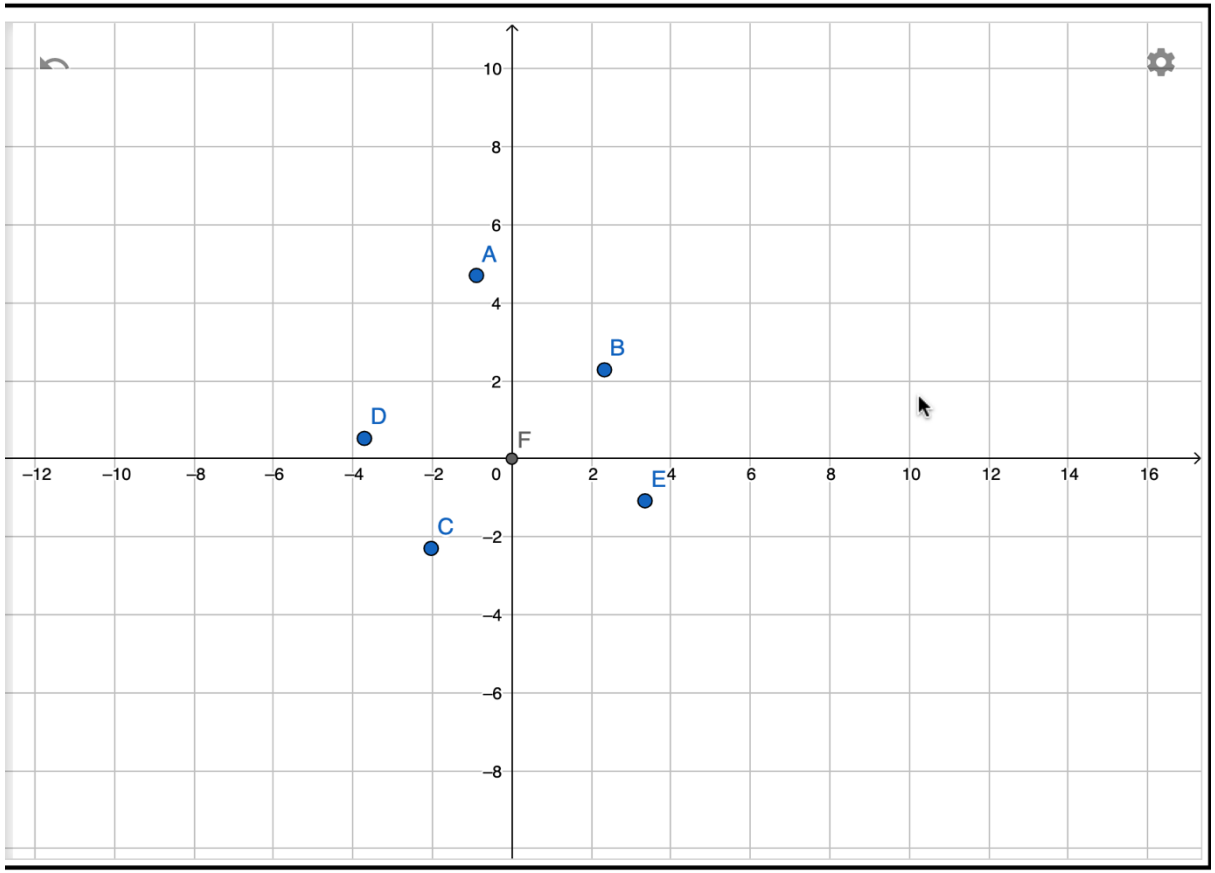
- For each point, the error is its distance to the nearest cluster
- To get WCSS(also called inertia) we square these errors and sum them.

$$WCSS = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - m_i||^2$$

Where:

- k = Number of clusters
- C_i = Set of all points in the i-th cluster
- x = Data point in cluster C_i
- m_i = Centroid of cluster C_i
- $||x - m_i||^2$ = Squared Euclidean distance between x and m_i

What if the number of centroids = number of samples



Reflection : What is WSSE when K is same as number of data points ?

Dunn Index

- It is calculated as the lowest inter-cluster distance (i.e. the smallest distance between any two cluster centroids) divided by the highest intra-cluster distance (i.e. the largest distance between any two points in any cluster)

- DI = (lowest inter cluster distance) / (highest intra cluster diameter)

- Dunn Index (to be maximized) tells us about quality of clustering, the more is better

- To maximize Dunn Index ,
  - maximize numerator term (well separated clusters) and
  - minimize denominator term(compact clusters)

Choosing number of clusters k

- The Elbow Method is one of the most popular methods to determine this optimal value of k.
- "elbow point"—the value of **k** where adding more clusters **does not significantly reduce WCSS**.
- You need to run the k-means clustering algorithm with different values of k.
- Record SSE for each experiment and find the Elbow point (point at which the *curve starts flattening*)

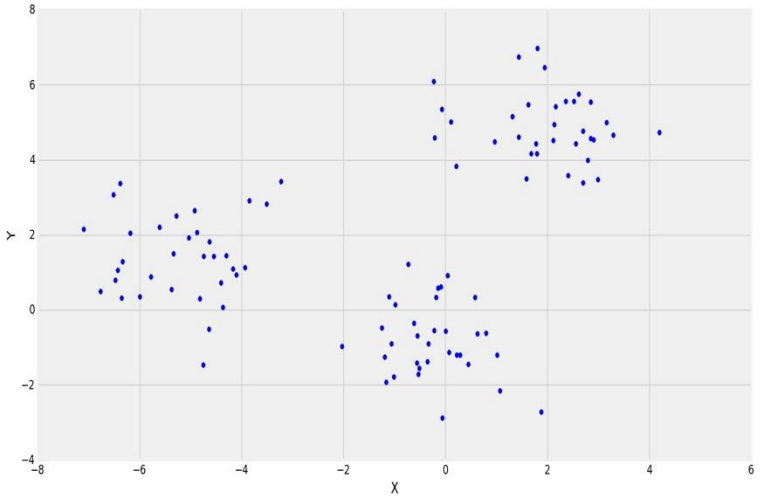


Figure: Distribution of training data in 2-D space

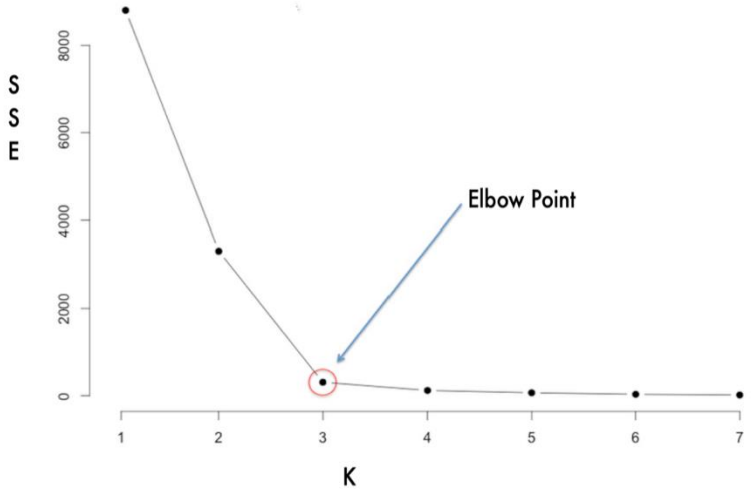


Figure: WSSE (Inertia) Plot

Choosing value of k …

- The intuition is that increasing the number of clusters will naturally improve the fit.
- but that, at some point this is over-fitting, and the elbow reflects this.
- For this graph clearly the elbow is forming at K=3. So the optimal value will be 3 for performing K-Means.

Leader clustering algorithm

- There are also algorithms for adding new centres *incrementally* or deleting empty ones.
- In, *leader cluster algorithm* an instance that is far away from existing centres causes the creation of a new centre at that point
- far away instance is defined as an instance that is beyond some predefined distance threshold.
- This is similar to ART algorithm in Neural Networks

Algorithm:

1. **Initialise threshold and the first Cluster centre (first leader)**
    1. Choose a **threshold distance (ε)** that determines when a new leader (cluster center) is created.
    2. The first data point becomes the **first leader (cluster center)**.
2. **Process Each New Data Point Sequentially**
    1. For each incoming point **x**, calculate its **distance** to all existing cluster leaders.
    2. **assign** the point **x** to the cluster leader from which it is at minimum distance.
    3. If the distance to all leaders is **greater than ε**, create a **new leader (new cluster center)** and assign the point to it.
3. **Repeat Until All Points Are Processed**

    Continue assigning points to existing clusters or creating new clusters based on the threshold distance (ε), till all the points are exhausted.

Example:

Find the clusters using Leader Clustering Algorithm with a threshold of 4.1 using Manhattan distance for the points:

{(1,1), (4,4), (7,7),(0,4), (4,0), (5,5), (9,9),(2,1)}.

ANS. Stepwise clustering process

Cluster	St-1	St-2	St-3	St-4	St-5	St-6	St-7	St-8
C1	**(1,1)**			(0,4)	(4,0)			(2,1)
C2		**(4,4)**				(5,5)		
C3			**(7,7)**				(9,9)	

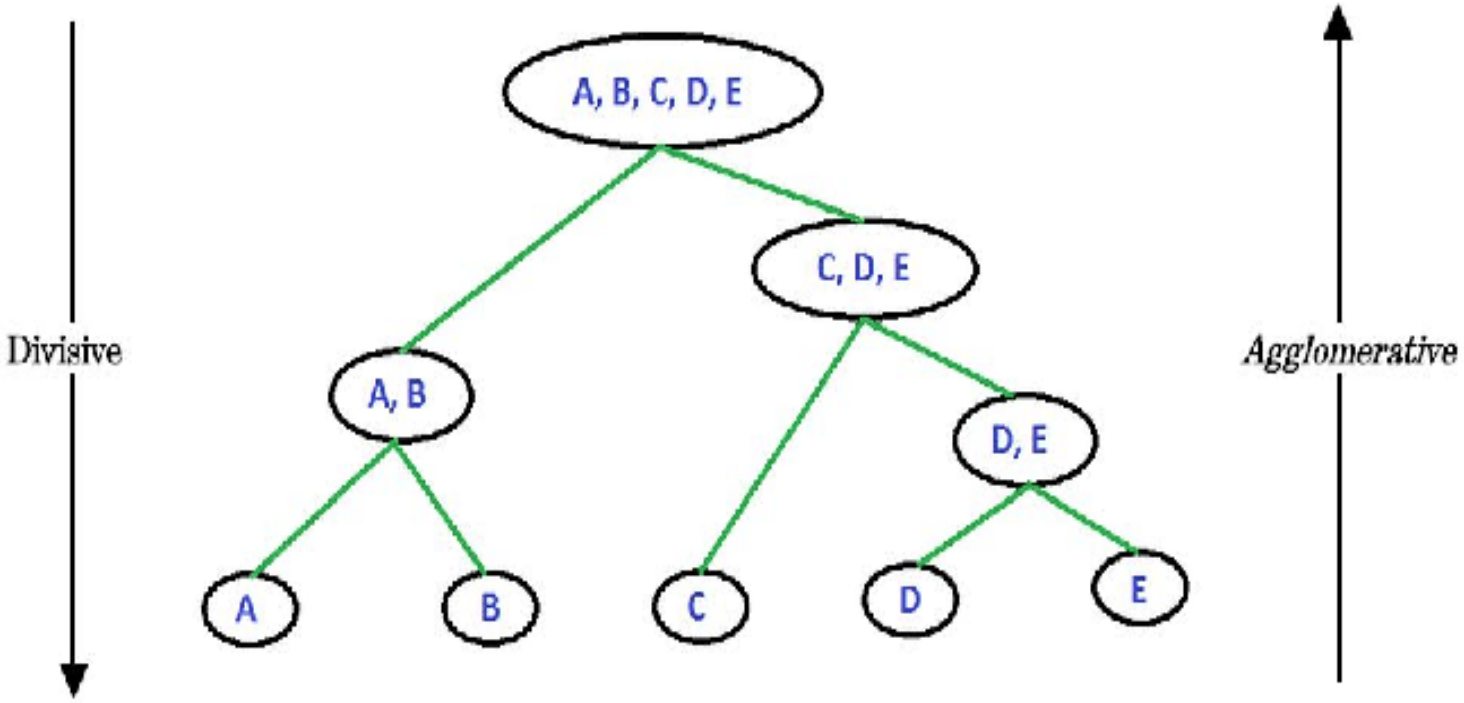
Hierarchical Clustering

- **Hierarchical clustering**, is an algorithm that groups similar objects into groups called clusters**, when the data is organised in a hierarchical manner, that is clusters have subclusters, and subclusters have sub subclusters, and so on**.
- The endpoint is a set of **clusters**, where each cluster is distinct from each other cluster, and the objects within each cluster are broadly similar to each other.

Hierarchical Clustering…

- There are **two types** of hierarchical clustering, Agglomerative and Divisive.
  - In ***agglomerative or bottom-up*** *clustering* each observation starts with itself as a cluster. Then, compute the distance between each of the clusters and join the two most similar clusters. Finally, repeat these steps until a predefined number of clusters (k) is reached.
  - In ***divisive or top-down*** clustering method we assign all of the observations to a single cluster and then partition the cluster to at least two similar clusters.

Two types of Hierarchical Clustering



Hierarchical Data and its Agglomerative Clustering

- The algorithm starts by creating one cluster for each input (data shown in yellow circles)
- Algorithm steps are shown as 1,2,3,4

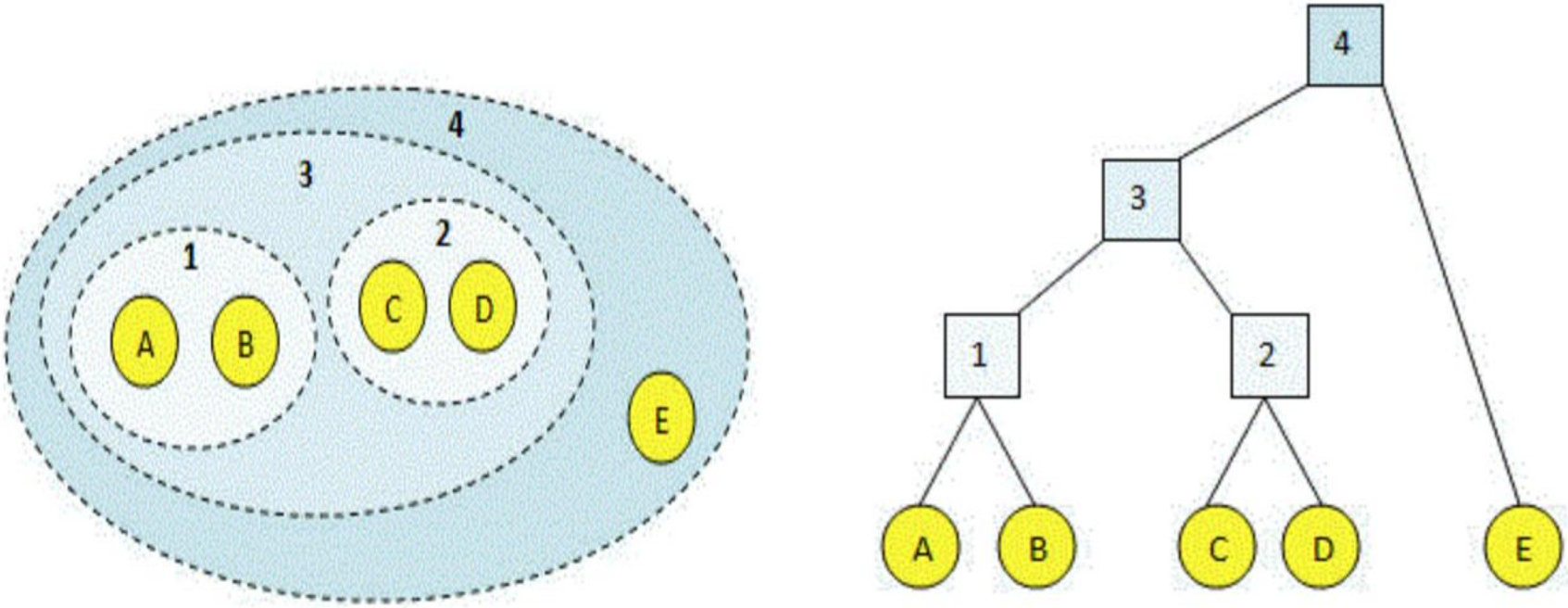
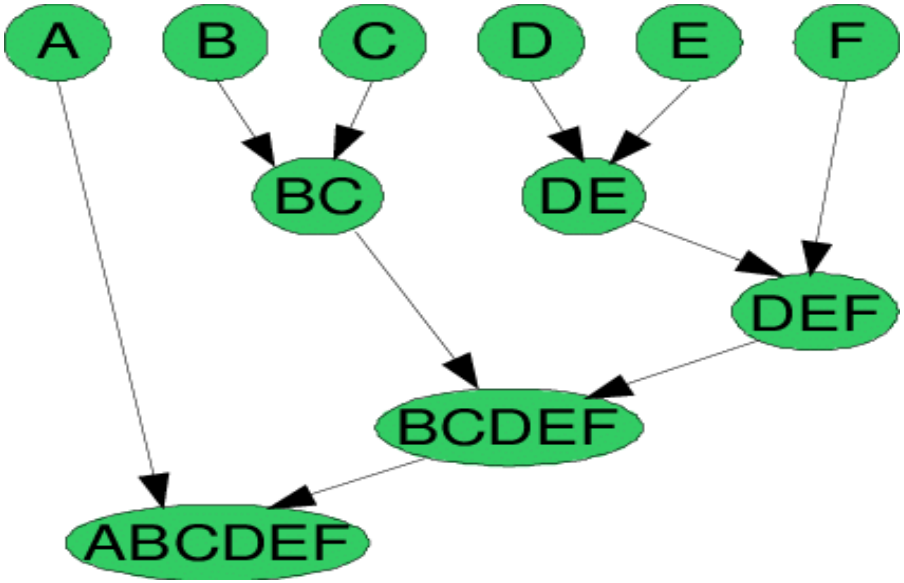


Figure: Left - Clustering Process, Right -Binary Tree Representation

Agglomerative Clustering

- Start with N groups each with one instance and *merge two closest groups at each iteration*



- Level one corresponds to N clusters and level N corresponds to one cluster
- Sample existing in the same cluster at level l will continue to stay together in the higher levels (a hierarchical sequence)

Ways to measure inter cluster distances:

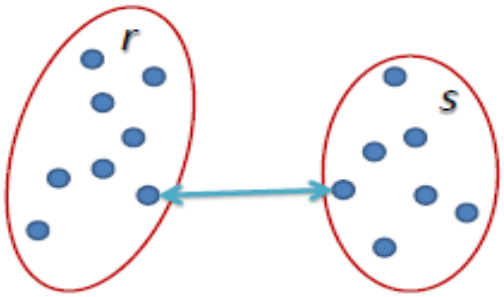
How to calculate the distance between two clusters in the case where a cluster contains more than one sequence(Linkage):

- **Single-linkage**: The distance between the two clusters is taken as the *smallest* distance between *all possible pairs of elements*
- **Complete-linkage**: The distance between the two clusters is taken as the *largest* distance between all possible pairs of elements
- **Average-linkage**: The distance between the two clusters is taken as the *average* distance between all possible pairs of elements
- **Centroid-Linkage**: It measures the distance between the centroids (means) of the two clusters. It only works with Euclidean distance.

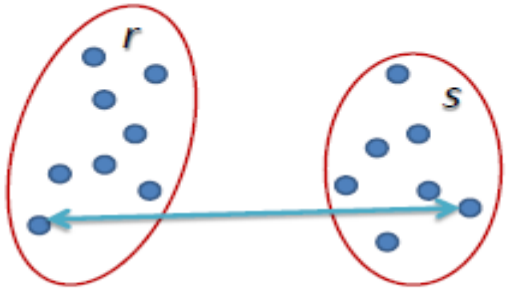
Comparison of Linkage Methods

Method	Uses Centroid?	Uses Point Distances?	Compatible with Non-Euclidean?
Single Linkage	❌ No	✅ Min pairwise	✅ Yes
Complete Linkage	❌ No	✅ Max pairwise	✅ Yes
Average Linkage	❌ No	✅ Avg pairwise	✅ Yes
Centroid Linkage	✅ Yes	❌ Uses mean vector	❌ Only Euclidean

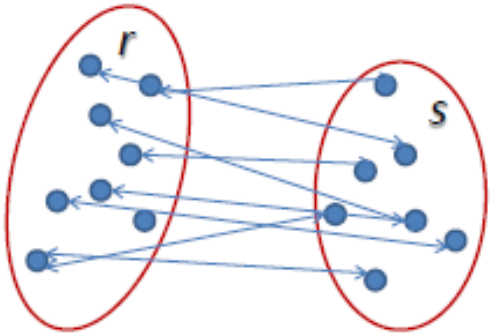
Single, Complete, Average and Centroid Linkage



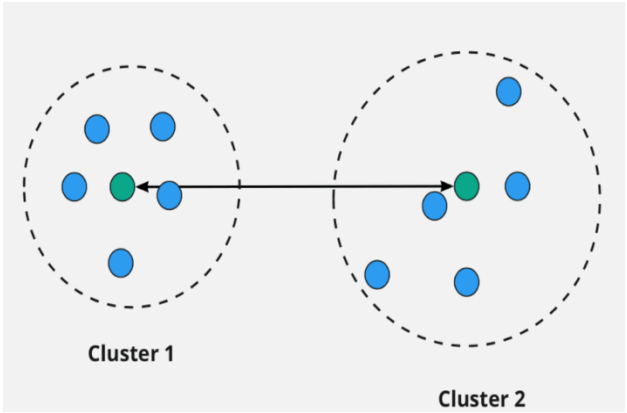
$$L(r,s) = \min(D(x_{ri}, x_{sj}))$$



$$L(r,s) = \max(D(x_{ri}, x_{sj}))$$



$$L(r,s) = \frac{1}{n_r n_s} \sum_{i=1}^{n_r} \sum_{j=1}^{n_s} D(x_{ri}, x_{sj})$$



Steps of Agglomerative Clustering

1. Initialize Each Data Point as a Cluster

- Begin with **n clusters**, where each data point is its own cluster.

2. Compute the Pairwise Distance Matrix

- Calculate the **distance** or **similarity** between every pair of clusters using a chosen distance metric (e.g., Euclidean distance, Manhattan distance).
- Represent these distances in a **distance matrix**.

3. Identify the Closest Pair of Clusters

- Identify the two clusters that are the **most similar** (or have the smallest distance between them).

4. Merge the Closest Clusters

- Combine the two closest clusters into a single cluster.
- Update the number of clusters by reducing it by one.

5. Update the Distance Matrix

- Recalculate the distances between the newly formed cluster and all remaining clusters. The way distances are updated depends on the linkage method used.

6. Repeat Steps 3 to 5

- Continue merging clusters and updating the distance matrix until one of the following stopping criteria is met:
    - A predefined number of clusters (kk) is reached.
    - A distance threshold is exceeded.

Typically used inter cluster distance measures

1. **Euclidean Distance (L2 norm)**:

$$d_{\text{Euclidean}}(A, B) = \sqrt{\sum_{i=1}^{n}(A_i - B_i)^2}$$

   - Measures the straight-line (shortest) distance between two points.

2. **Manhattan Distance (L1 norm)**: or City Block Distance

$$d_{\text{Manhattan}}(A, B) = \sum_{i=1}^{n}|A_i - B_i|$$

   - Measures distance along grid-like paths (e.g., city blocks).

3. **Chessboard (Chebyshev) Distance (L∞ norm)**:

$$d_{\text{Chessboard}}(A, B) = \max_i |A_i - B_i|$$

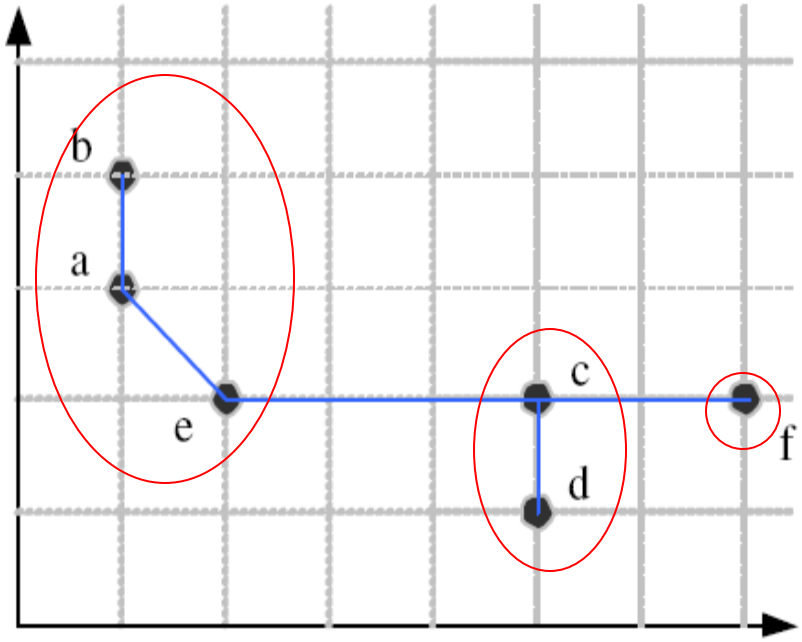
   - Measures the maximum movement needed in any single dimension (used in chess for king's moves).

Dendrogram

- A **dendrogram** is a plot showing a tree-like structure that records the sequences of merges or splits in hierarchical clustering.
- It is most commonly created as an output from **hierarchical clustering**.
- We can use a dendrogram to visualize the history of groupings and figure out the optimal number of clusters.
- The **x-axis** represents the **individual data points** or **clusters**.
- The **y-axis** represents the **distance or dissimilarity** between the clusters being merged.

Example: Single-Link Clustering

Dataset=[[1,3],[1,4], [5,2], [5,1], [2,2], [7,2]]



Graph is between observations and the inter cluster distances

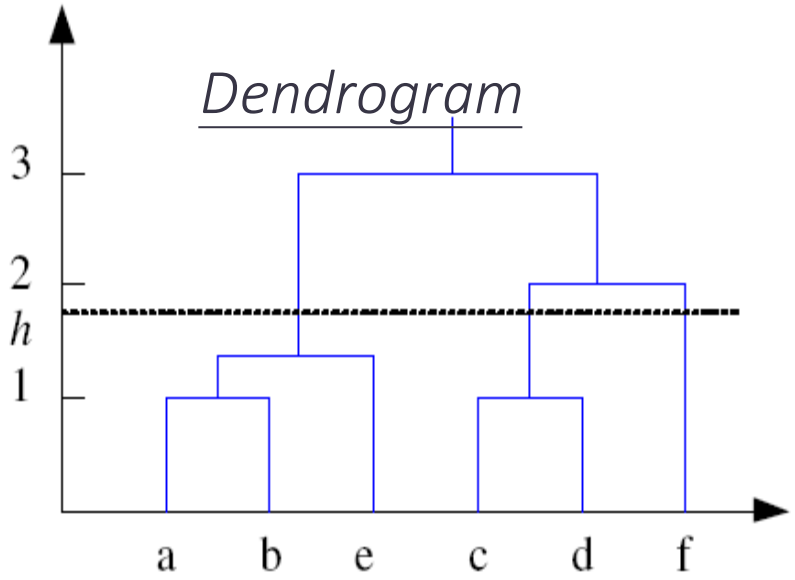
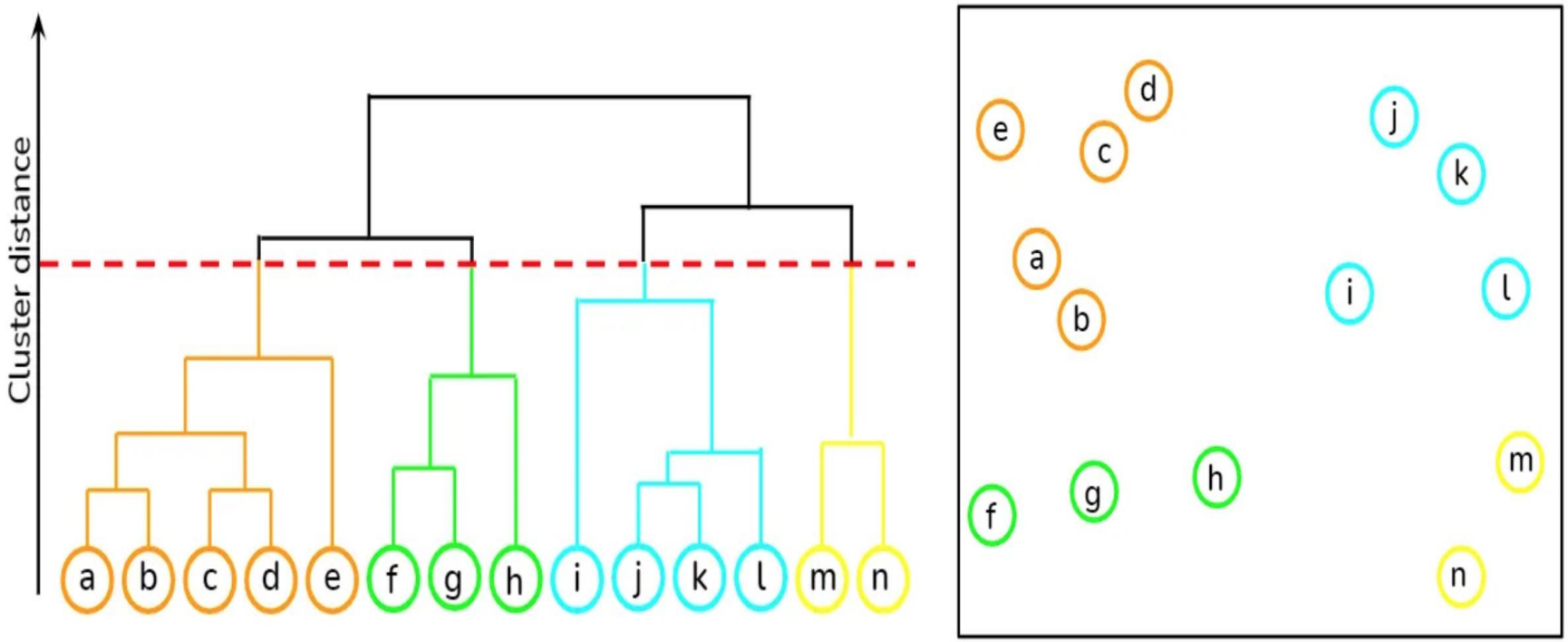


Figure A two-dimensional dataset(left) and the dendrogram(right). The tree can be intersected at a desired value of h to get the clusters

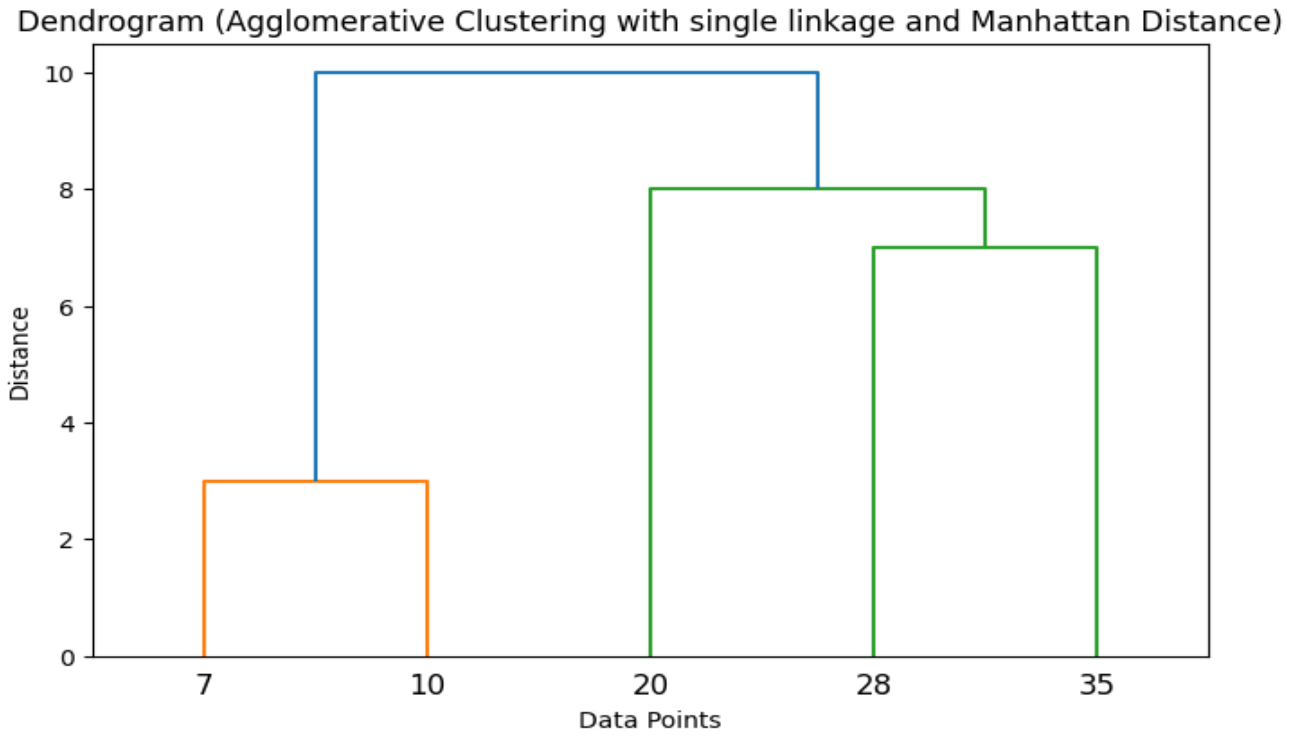
Sliced dendrogram



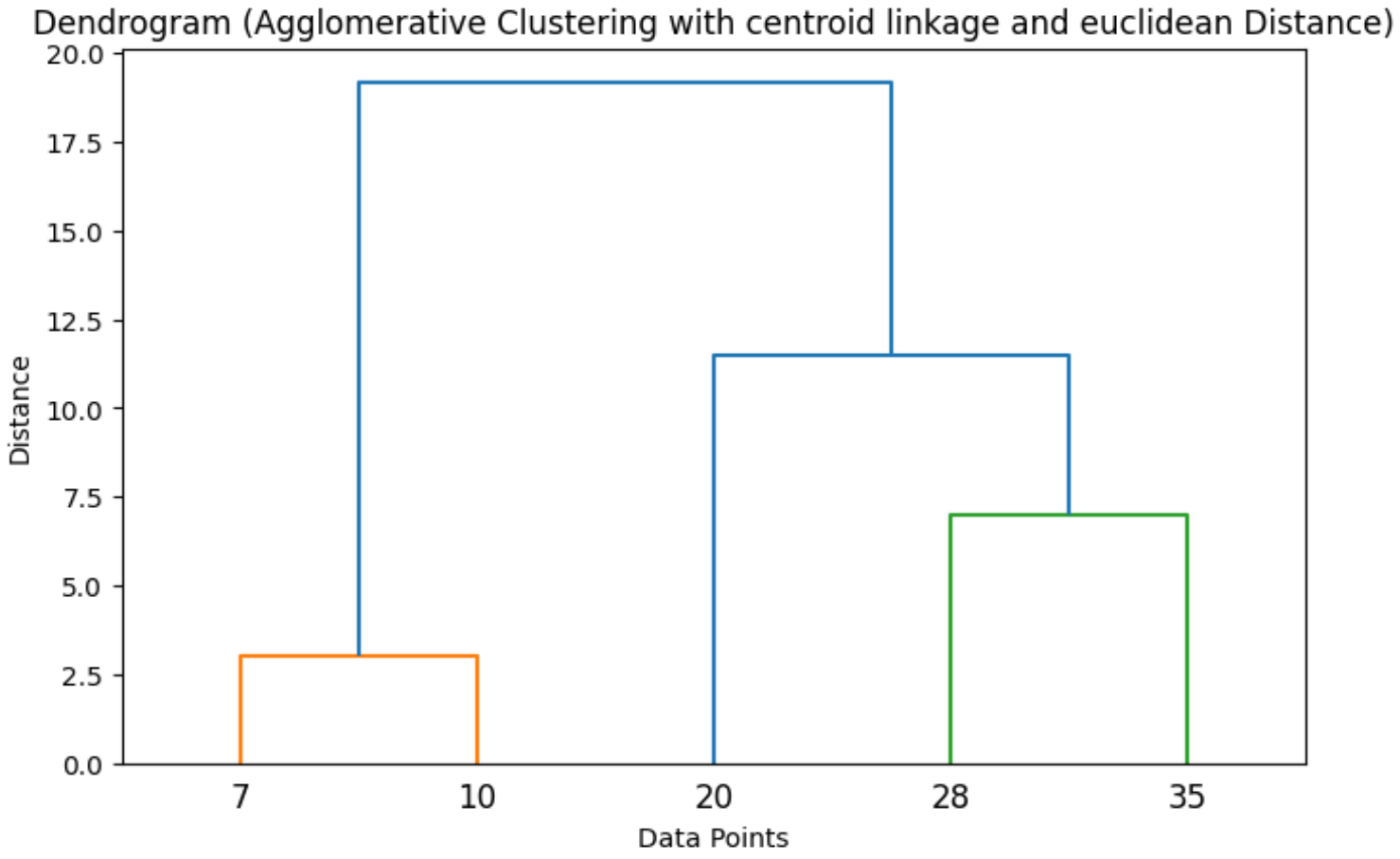
A sliced dendrogram with its resulting clusters marked in its corresponding feature vector space on the right side

Example-1

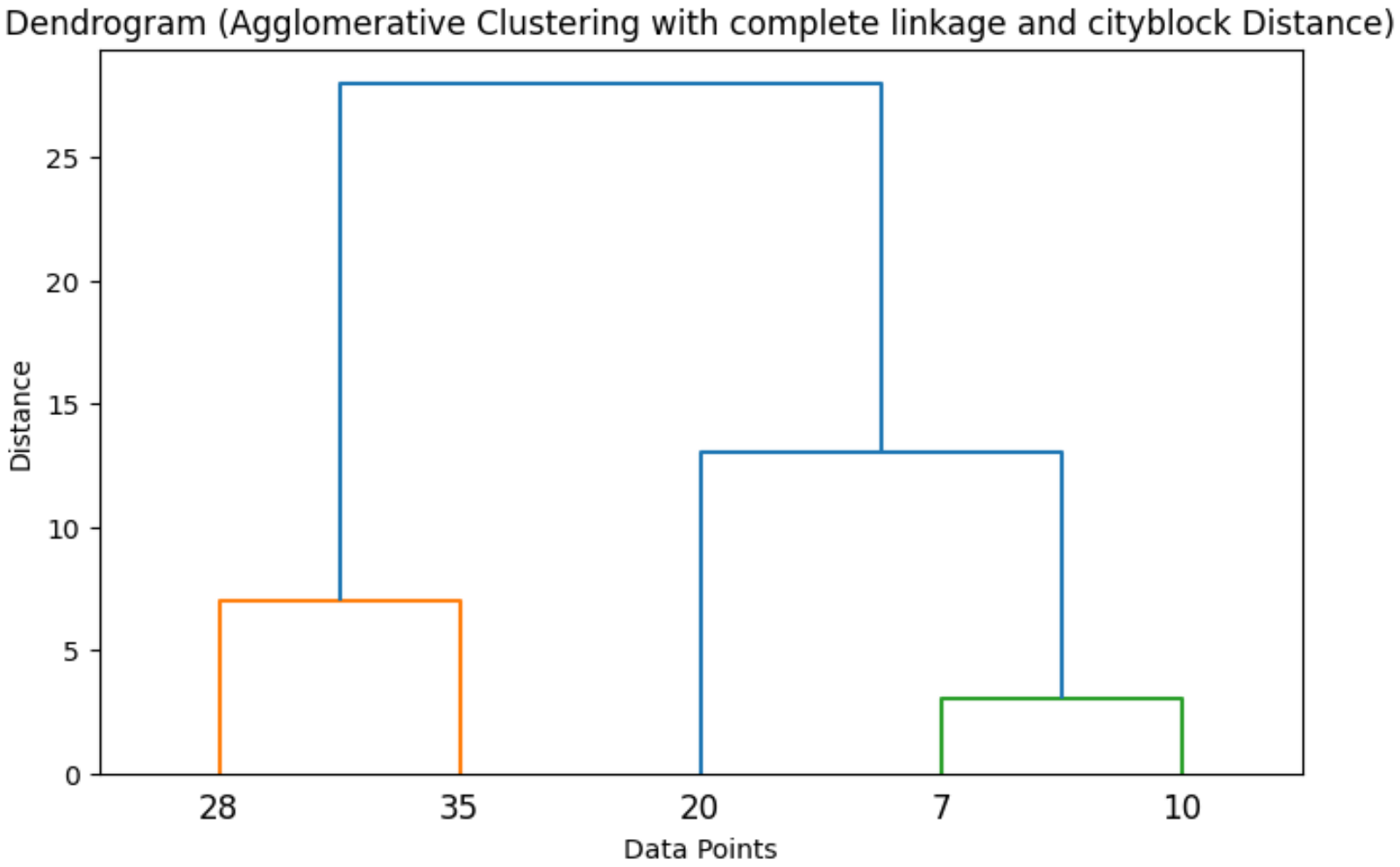
For the one-dimensional data set **{7,10,20,28,35}**, perform Agglomerative clustering with single linkage and plot the dendrogram to visualize it using City block distance.



Example-2: For the one-dimensional data set **{7,10,20,28,35}**, perform Agglomerative clustering with centroid linkage and plot the dendrogram to visualize it using Euclidean distance.



With complete linkage



Code for Agglomerative clustering

```
import numpy as np
import scipy.cluster.hierarchy as sch
import matplotlib.pyplot as plt

# Given one-dimensional dataset
data = np.array([[7], [10], [20], [28], [35]]) # Convert
to 2D array
linkage = 'centroid'
dm = 'euclidean'
# Compute linkage matrix using Manhattan distance
Z = sch.linkage(data, method=linkage, metric=dm)

# Plot dendrogram
plt.figure(figsize=(8, 5))
sch.dendrogram(Z, labels=[7, 10, 20, 28, 35],
color_threshold=10) # Adjust color threshold for 2
clusters
plt.title(f"Dendrogram (Agglomerative Clustering with
{linkage} linkage and {dm} Distance)")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```

Reflection

1. What is the key difference between Agglomerative and Divisive Clustering?

a) Agglomerative starts with one cluster, while Divisive starts with individual points.
b) Agglomerative is a bottom-up approach, while Divisive is a top-down approach.
c) Agglomerative requires a predefined number of clusters, but Divisive does not.
d) Agglomerative is always more efficient than Divisive.

2. What is the stopping criterion in Agglomerative Clustering?

a) When the clusters stop moving.
b) When a predefined number of clusters is reached.
c) When clusters stop merging.
d) When the model achieves zero error.

Reflection

3. In which type of clustering method does a dendrogram help visualize the clustering process?

a) K-Means Clustering
b) DBSCAN
c) Hierarchical Clustering
d) Gaussian Mixture Model

4. In Leader Clustering, a new data point is assigned to a cluster if its distance to that cluster is:

a) Greater than a predefined threshold (ε)
b) Less than a predefined threshold (ε) and is minimum across all the existing leaders
c) Less than a predefined threshold (ε) and is maximum across all the existing leaders
d) Greater than a predefined threshold (ε) and is maximum across all the existing leaders

Homework

1. Perform Average link clustering on the following data with city block distance **[7,10,20,28,35].**
2. Perform single link clustering on the following 2-D data with city block distance and draw the resulting dendrogram.

   [[1,3],[1,4],[5,2],[5,1],[2,2],[7,2]]
3. Perform Centroid link clustering on the above data and draw the resulting dendrogram.
4. Perform single link clustering on the following data with city block distance **[7,10,20,28,35], for k=2.**

Semi Supervised Learning

- **Semi-supervised learning** is an approach to machine learning that combines a small amount of labeled data with a large amount of unlabelled data during training.
- Semi-supervised learning falls between unsupervised learning (with no labelled training data) and supervised learning (with labelled training data).
- It is based on the continuity assumption i.e. *Points that are close to each other are more likely to share a label.*
- When we need to build a supervised learning model but don't have sufficient labelled samples in our data, semi supervised learning is useful.
- idea is to build a supervised learning model based on the output of the unsupervised learning process.

Steps of Semi-Supervised Classification

1. **Training a Classifier on a Small Labelled Dataset**: A supervised model is trained using a limited number of labelled examples.
2. **Unsupervised Learning on a Larger Unlabelled Dataset**: A clustering algorithm (e.g., K-Means) is used to group unlabelled data points.
3. **Classifying Cluster Centroids**: The trained classifier is used to predict the labels of the cluster centroids.
4. **Label Propagation**: Once centroids are classified, their labels are propagated to all data points in the corresponding cluster.

Example

- One real world application of semi-supervised learning, is ***webpage classification.***
- Suppose you want to classify any given webpage into one of several categories (like "Educational", " Shopping", ”Music”, “Tourism”, “Technology”, etc.).
- This is a case where it's expensive to go through tens of thousands of webpages and have humans annotate them.
- But suppose you have a few labelled pages in each category
- Initially clustering is done on the large number of unlabelled webpages to obtain cluster centres.
- Cluster centres could be labelled using KNN, with these few labelled pages, as if the CCs are new examples to be classified.
- Features of any new page are compared with the cluster centres(now labelled), and the label is found using nearest centroid classification .

SS learning using Sklearn

```
from sklearn.semi_supervised import LabelPropagation
from sklearn.datasets import make_classification
import numpy as np
# Generate synthetic dataset
X, y = make_classification(n_samples=200, n_features=5, n_classes=3, random_state=42)
y[:150] = -1  # Mark 150 samples as unlabeled (-1)
# Apply LabelPropagation
lp_model = LabelPropagation(kernel='knn')
lp_model.fit(X, y)
# Predict labels for all samples
y_pred = lp_model.predict(X)
```

Parameters in Machine Learning

- In machine learning, a model M with parameters and hyper-parameters looks like, $Y \approx M_{\mathrm{H}}(\Phi|D)$

  where Φ are model parameters and H are hyper-parameters. D is training data and Y is output data (class labels in case of classification task).

- In a machine learning model, there are 2 types of parameters:
  - **Model Parameters:** These are the parameters in the model that must be determined using the training data set. These are the fitted parameters and fixed at the end of training.
  - **Hyperparameters:** These are adjustable parameters that must be tuned to obtain a model with optimal performance. These are fixed prior to training.

Continued…

- **Example-1**: simple **linear regression model based on OLS regression** using an m-dimensional training dataset.

$$\hat{y}_i = \sum_{j=0}^{m} X_{ij} w_j$$

Here w_j are model parameters, whereas there are no hyper parameters. In case of gradient descent based model, learning rate is an hyperparameter.

- **Example-2**: **k-means clustering:**
  - **Hyperparameters**: k, seed/initial values for cluster centres, distance measures
  - **Model Parameters**: final cluster centres

Parametric Models

- A learning model that summarizes data with a set of parameters of fixed size (independent of the number of training examples) is called a parametric model.
- The algorithms involve two steps:
  - Select a form for the function.
  - Learn the coefficients for the function from the training data.
- **Example: A linear regression model:**
- Let us take a 2-D problem where the mapping function is a line of the form

$$b_0 + b_1 * x_1 + b_2 * x_2 = 0$$

- Where b0, b1 and b2 are the coefficients of the line and serve as model parameters.
- **The problem**:
  - the actual unknown underlying function may not be a linear function like a line.
  - In case the assumption is wrong, the approach will produce poor results.

More Examples of Parametric Models

- Linear classification
- Logistic Regression
- Naive Bayes
- Linear Discriminant Analysis
- Perceptron
- Neural Networks

Nonparametric Machine Learning Models

- Algorithms that *do not make strong assumptions* about the form of the *relationship between input variables and the output variable or underlined distribution* of data, are called nonparametric machine learning algorithms.
- the number of parameters is not fixed before training and changes dynamically with data size
- Non-parametric models are well-suited for capturing non-linear relationships between input variables and the target variable, making them versatile for various data types and domains.
- Examples: K-Nearest Neighbours (KNN), Decision Trees, k-means clustering

Web links

- https://towardsdatascience.com/customer-segmentation-using-k-means-clustering-d33964f238c3

Thank You !